Methodology: Reinforcement Learning and Statistical Modeling for U.S. Equity Markets

Jackson Rusconi, Vanya Murzin, Jamie Dinh

October 28, 2025

1 Methodology

1.1 Overview

This study integrates statistical, machine learning, and reinforcement learning methods to analyze and predict U.S. stock market performance. Our primary objective is to assess how different modeling approaches handle time-series data in equity markets and how effectively they can predict price direction and returns. Each model class-statistical, machine learning, and reinforcement learning—offers a distinct balance between interpretability, flexibility, and computational complexity. By comparing across these paradigms, we aim to identify approaches that maintain robustness across various market conditions.

The methodology includes:

- **Linear Regression:** A simple baseline model for continuous price prediction.
- **Logistic Regression:** A classification model for predicting price direction.
- **ARIMA:** A classical time-series model capturing temporal autocorrelations.

- **Random Forest:** A tree-based ensemble for nonlinear pattern detection.
- **Neural Network:** A deep learning model that captures higher-order interactions.
- **Reinforcement Learning:** Advanced agents (RRL and PPO) for decision-based trading.

All models are evaluated on the same daily OHLCV dataset used in the exploratory data analysis (EDA), with engineered technical indicators and a walk-forward split for fair comparison.

1.2 Data, Features, and Splits

We reused the cleaned dataset from the EDA phase, containing daily open, high, low, close, and volume (OHLCV) data for eleven U.S. equities (AAPL, AMD, CSCO, MSFT, TXN, EMR, GE, HON, IBM, NVDA, UPS). Feature engineering included the following technical indicators: log returns, moving averages (MA7, MA30), 20-day rolling volatility, RSI14, and volume deltas. Categorical variables, such as trading day of the week and sector classification, were one-hot encoded. All features were standardized using only the training-set statistics to prevent data leakage.

The chronological data split followed a walk-forward scheme:

- **Training:** Start of history to 2016-12-31
- **Validation:** 2017-01-01 to 2019-12-31
- **Testing:** 2020-01-01 onward (COVID-era and recent data)

To determine the suitability of the data for time-series modeling, a stationarity analysis was conducted on the combined stock market dataset following exploratory data analysis. Stationarity testing is a critical diagnostic step for selecting appropriate forecasting models

such as ARIMA or ARMA, which assume that the statistical properties of the data (mean, variance, and autocorrelation) remain constant over time.

Figure 1 shows the overall time series of closing prices for all ten companies. The visual trend indicates clear long-term upward movement with noticeable volatility clustering, particularly around major market events such as the dot-com bubble (2000), the financial crisis (2008), and post-2020 market fluctuations. The continuous upward drift suggests the data is non-stationary, as both mean and variance appear to vary across time.

Figure 1: Time Series Analysis

To further assess stability, a rolling mean and rolling standard deviation were computed using a 30-day window (Figure 2). The red line representing the rolling mean fluctuates considerably across the time axis, while the dashed black line showing rolling standard deviation widens and narrows at different intervals. These patterns confirm that the time series lacks a constant mean and variance—characteristics of a non-stationary process.

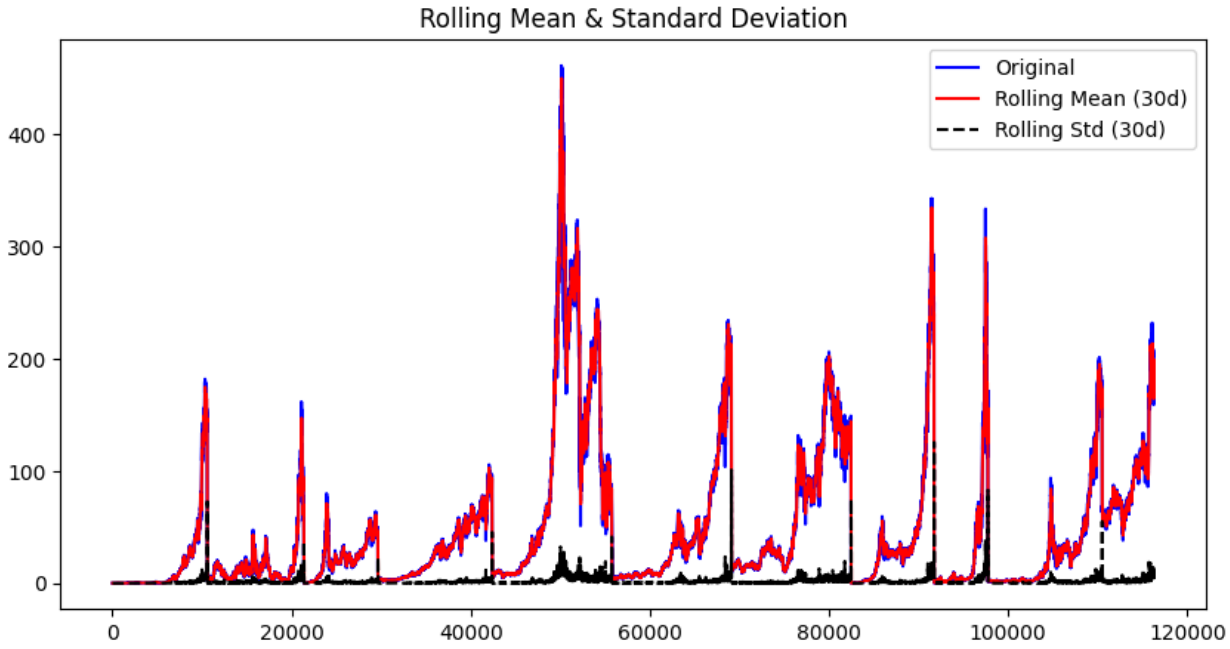

Figure 2: Rolling Mean and Standard Deviation of Stocks

To complement the visual findings, two formal statistical tests were applied: the **Augmented Dickey-Fuller (ADF)** test and the **Kwiatkowski–Phillips–Schmidt–Shin (KPSS)** test. The ADF test checks for the presence of a unit root, where the null hypothesis assumes *non-stationarity*, while the KPSS test assumes *stationarity* under its null hypothesis.

ADF Test Results

- ADF Statistic: -4.575107184304345
- p-value: 0.00014352610251783756
- Critical Value (1%): -3.430
- Critical Value (5%): -2.862
- Critical Value (10%): -2.567

- **Stationary (reject H_0)**

Because the p-value is less than 0.05 and the test statistic is below the critical threshold, the ADF test rejects the null hypothesis, suggesting that the series may be **stationary** after accounting for short-term dependencies.

KPSS Test Results

- KPSS Statistic: 3.797
- p-value: 0.010
- **Non-stationary (reject H_0)**

Given that the p-value is below 0.05, the KPSS test rejects its null hypothesis, indicating that the data is **non-stationary**. Together, these results highlight mixed evidence: while ADF suggests short-term stationarity, the KPSS indicates long-term non-stationarity, consistent with typical financial time-series behavior.

A conflict in the results for both tests indicates that the stock data is slightly stationary in the short term (ADF Test) and non stationary for the long term. This indicates mean changes over time despite some mean-reverting tendencies. In other words, while short-term market movements may appear to oscillate around an average level, these averages themselves shift over time due to evolving market forces, such as changes in investor sentiment, macroeconomic conditions, and corporate performance. This is common in financial time series, especially stock prices, which often follow a random walk pattern in which future price movements are largely independent of past movements, with each new price reflecting all available information in the market at that moment.

This behavior violates the assumptions of stationary time-series models like ARMA. Therefore, before applying forecasting techniques, differencing or transformation must be applied to stabilize the mean and variance.

Future modeling will require that first-order differencing:

$$\Delta\text{Close}_t = \text{Close}_t - \text{Close}_{t-1}$$

be employed to remove trend components and achieve approximate stationarity. This adjustment will enable the implementation of ARIMA$(p, 1, q)$ models to capture both autoregressive and moving-average relationships in the differenced series while preserving the underlying market dynamics.

1.3 Statistical Models

1.3.1 Linear Regression

Linear regression served as the simplest predictive model for continuous next-day price values. It establishes a direct relationship between the dependent variable (price or return) and independent predictors (technical indicators). The model can be expressed as:

$$Y_t = \beta_0 + \beta_1 X_{1,t} + \beta_2 X_{2,t} + \cdots + \beta_n X_{n,t} + \epsilon_t, \tag{1}$$

where Y_t is the predicted value, X_i represents independent variables such as MA7, RSI14, and volatility, and ϵ_t is the residual. Model parameters were estimated via ordinary least squares (OLS). Performance was assessed through RMSE, MAE, and R^2, while Figure 3 displays residual and predicted-actual comparisons. The linear model provides interpretability and serves as a transparent benchmark for later models.

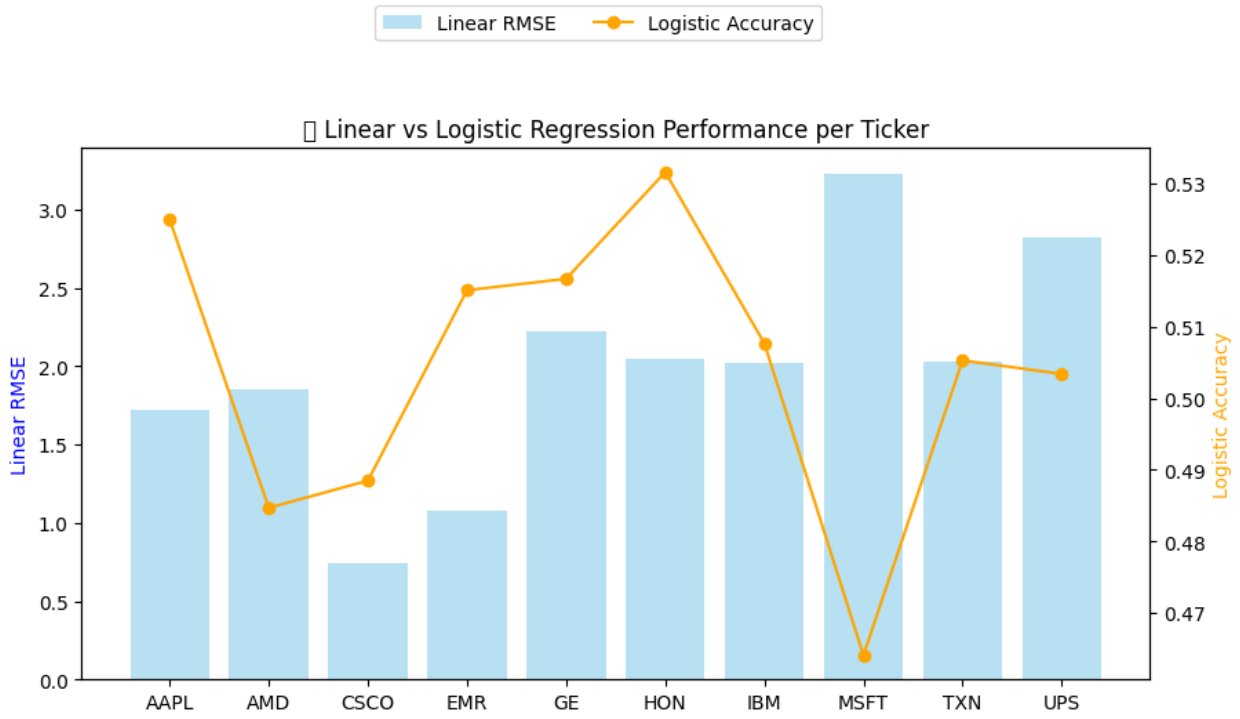

Figure 3: Regression Models Performance on Tickers

1.3.2 Logistic Regression

Logistic regression was employed to predict directional changes in stock prices—whether the next-day close would be higher or lower. The logistic function models the probability that a stock's return is positive:

$$P(Y_t = 1) = \frac{1}{1 + e^{-(\beta_0 + \sum_i \beta_i X_{i,t})}}. \tag{2}$$

Model parameters were optimized using maximum likelihood estimation (MLE). Metrics such as accuracy, precision, recall, F1 score, and ROC-AUC were used to evaluate model performance. The confusion matrix highlights predictive bias and misclassification trends. This probabilistic approach captures the binary structure of trading decisions and enhances interpretability regarding feature contributions.

1.3.3 ARIMA (Autoregressive Integrated Moving Average)

ARIMA was implemented to capture temporal dependencies and autocorrelation structures in financial time-series data [8]. ARIMA(p, d, q) combines autoregressive (p), differencing (d), and moving-average (q) components as:

$$Y_t = c + \sum_{i=1}^{p} \phi_i Y_{t-i} + \sum_{j=1}^{q} \theta_j \epsilon_{t-j} + \epsilon_t. \tag{3}$$

Parameters (p, d, q) were selected by minimizing the Akaike Information Criterion (AIC). Differencing ensured stationarity $(d \geq 1)$, while residual diagnostics validated model adequacy. Rolling-window forecasts were produced, and positions were determined by the sign of one-step-ahead return predictions.

The Moving Average (MA) component, on the other hand, models the relationship between an observation and past error terms (or shocks). This captures how random disturbances in previous periods continue to influence current values. For instance, unexpected market events or volatility spikes from earlier days can still impact today's price. Mathematically, this is represented as:

$$Y_t = \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \epsilon_t,$$

where θ_i are the moving average parameters and ϵ_t is the current residual.

Finally, the Integrated (I) component addresses non-stationarity by differencing the time series. Instead of modeling raw prices, which often contain trends or seasonal effects, the model uses transformed values such as daily price changes. This process removes long-term trends, allowing the resulting series to exhibit a constant mean and variance over time, making it suitable for ARMA modeling.

This combination enables ARIMA models to capture both the memory and the random-

ness inherent in time-dependent data, making them highly effective for short-term forecasting in financial and economic contexts.

1.4 Machine Learning Models

1.4.1 Random Forest

The Random Forest algorithm is an ensemble-based learning method that constructs a collection of decision trees to enhance predictive accuracy and reduce overfitting. Each tree is trained on a random subset of data and features, and the ensemble's prediction is derived from the aggregated results of all trees. This technique, introduced by Breiman [2], builds upon the concept of random subspace methods proposed by Ho [4]. Random Forests are particularly suited to stock market predictions due to their ability to model nonlinear relationships, handle high-dimensional features, and maintain robustness in the presence of noise. In stock market data, where technical indicators such as moving averages, relative strength index (RSI), and volume interact, Random Forests effectively capture dependencies that linear models may overlook.

The prediction is computed as:

$$\hat{Y}_t = \frac{1}{B} \sum_{b=1}^{B} f_b(X_t), \tag{4}$$

where B is the number of trees and $f_b(X_t)$ denotes the b-th tree's prediction. Hyperparameters such as the number of trees, tree depth, and minimum leaf size were tuned via cross-validation on the validation split.

The first Random Forest Model we looked at was a regression model. This model had strong predictive power. The R-squared value was .9985, in other words, the independent variables captured 99.85% of the variance of the stock market movement. The Mean Absolute Percentage Error seems to be fairly good as well from the model. The regression model

heavily relied on the close feature for the model.

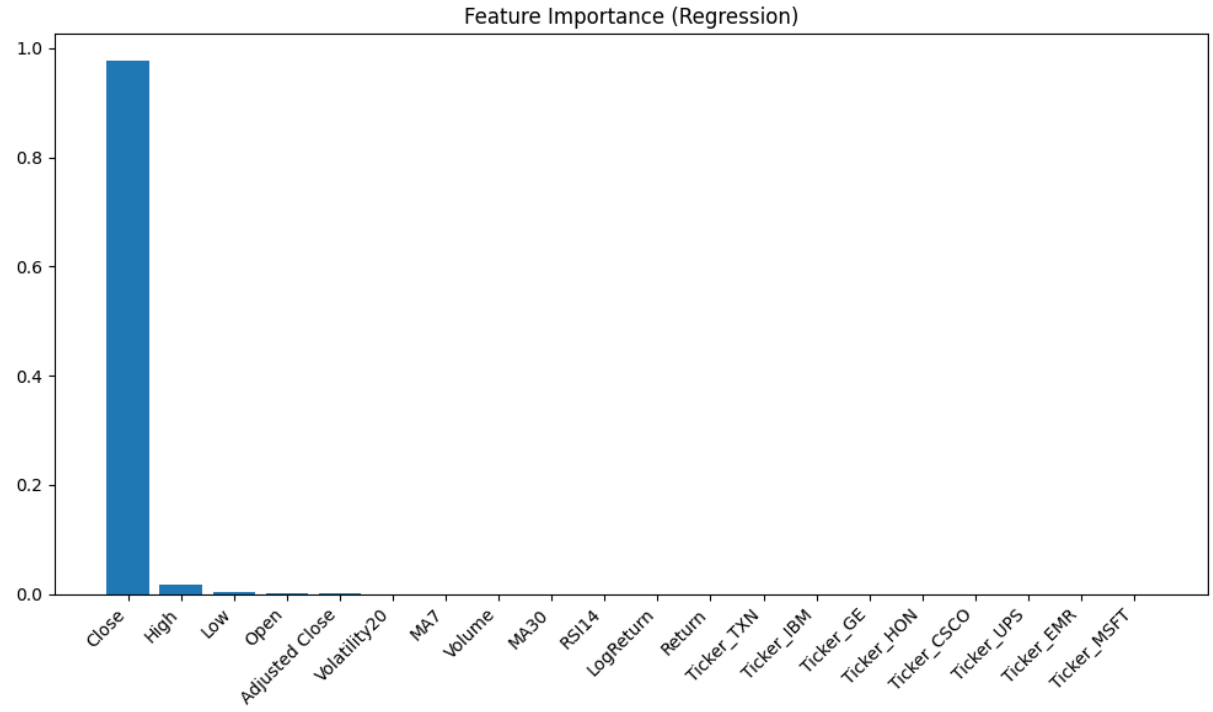

Figure 4: Important Features on Regression Model

The classification model was a bust. The accuracy measures for the classification determined whether a stock would go upward or downward in price from the previous day. The accuracy sat around 50%. In other words, you had a slightly better chance of picking whether the stock would go up or down than the model did. There were also many Type I and Type II errors in the model. This model might not be a good one to explore any further. What was interesting about the classification model was that it didn't rely heavily on one feature and instead used many different features in its model.

```
=== RandomForest Classification ===
Accuracy: 0.4985  Precision: 0.5209  Recall: 0.4202
ROC AUC : 0.5042
```

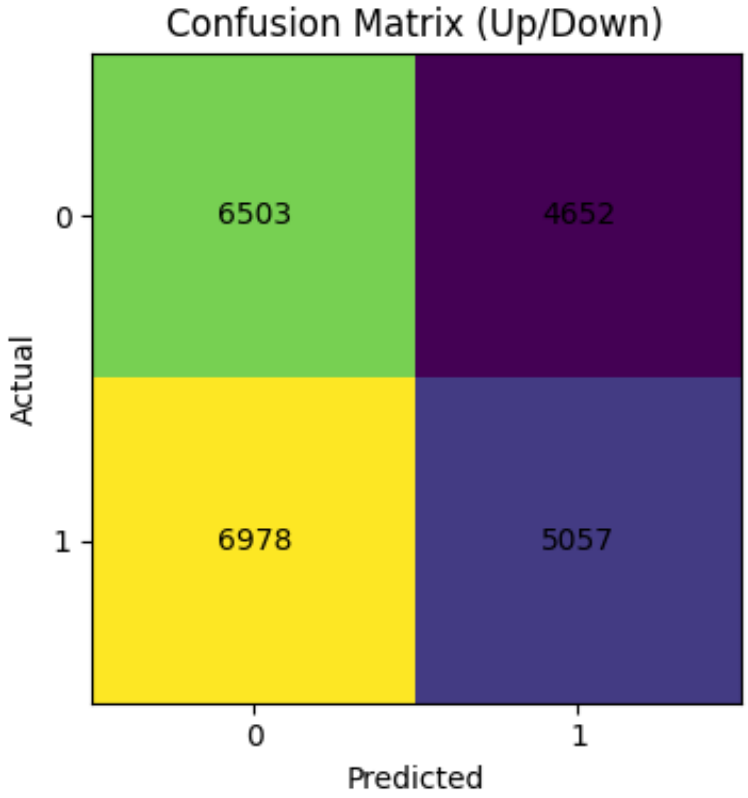

Figure 5: Classification Model Performance

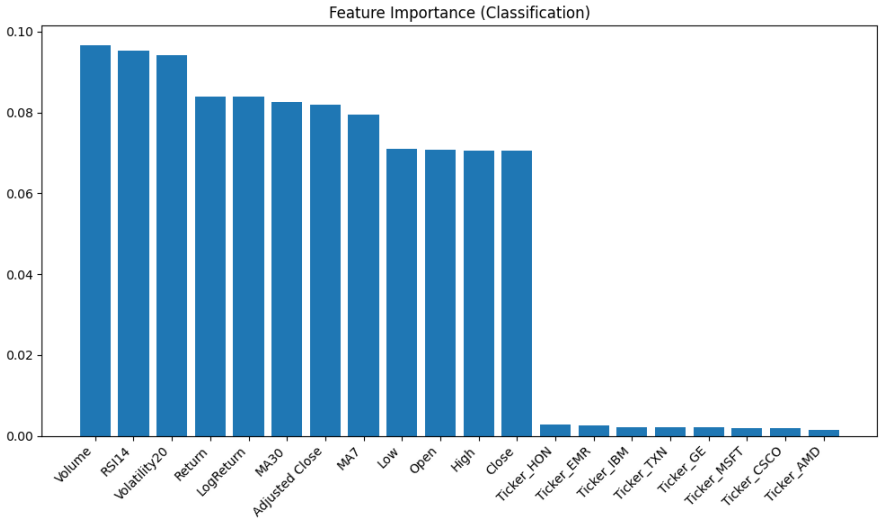

Figure 6: Important Features on Classification Model

1.4.2 Neural Network

Artificial Neural Networks (ANNs) are nonlinear predictive models inspired by the structure of the human brain. They consist of interconnected neurons organized in layers that transform input data through weighted combinations and nonlinear activation functions. Training occurs through backpropagation and gradient descent, adjusting network parameters to minimize prediction error [9].

Neural Networks are advantageous in financial forecasting because they can learn complex, hierarchical representations and capture nonlinear interactions among market variables. This allows us to do way more than simple predictions like the Random Forest Models. It helps look at many different factors affecting risk, profitability and accuracy. Foundational works in deep learning [3] have demonstrated their scalability and adaptability, and prior studies have validated their success in stock market forecasting contexts [1, 5].

The model learns nonlinear mappings:

$$h^{(l)} = g(W^{(l)}h^{(l-1)} + b^{(l)}), \tag{5}$$

where $g(\cdot)$ is the activation function.

The model's inclusion allows for direct comparison between traditional ensemble learning and modern deep learning techniques within a consistent experimental framework. Technical indicators created by feature engineering were used in order to create equations designed to simultaneously prevent risk and increase profitability. We adjusted the learning rate to .0001 in order to prevent overfitting.

1.5 Reinforcement Learning Models

1.5.1 Recurrent Reinforcement Learning (RRL)

Recurrent Reinforcement Learning (RRL) directly optimizes trading strategies by maximizing a differentiable Sharpe ratio objective rather than predicting future prices [7]. At each step, a recurrent policy (e.g., GRU-based) outputs a pre-action $u_t \in [-1, 1]$ converted into a discrete position $a_t \in \{-1, 0, +1\}$. The policy parameters θ are optimized to maximize

$$J(\theta) = \frac{\mathbb{E}[r_t]}{\sqrt{\mathbb{E}[r_t^2] - (\mathbb{E}[r_t])^2}}, \tag{6}$$

where r_t denotes trade returns. Gradients $\frac{\partial J}{\partial \theta}$ are computed through backpropagation-through-time, updating parameters as

$$\theta_{t+1} = \theta_t + \eta \frac{\partial J}{\partial \theta_t}. \tag{7}$$

To stabilize learning, exponential moving averages approximate mean and variance of returns, forming the differential Sharpe reward. A GRU with 32–64 hidden units balances expressiveness and regularization.

1.5.2 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a policy-gradient reinforcement learning method designed for stability in nonstationary environments [6]. It constrains updates via a clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right], \tag{8}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\hat{A}_t$ denotes advantage estimates. The environment defines states $s_t = x_t$, discrete actions $a_t \in \{-1, 0, +1\}$, and trading rewards $r_t^{\text{trade}} = p_{t-1} r_t^{\text{asset}} - c|\Delta p_t|$, where c is transaction cost. PPO will be trained using a learning rate 3×10^{-4}, clipping threshold 0.2, GAE $\lambda = 0.95$, entropy coefficient 0.01, and batch size 64.

1.6 Regularization and Robustness

To prevent overfitting and improve generalization, we will apply:

- Walk-forward retraining for all models.
- Mild volatility/drawdown penalties for RL rewards.
- Expanding-window feature standardization.
- Cost-aware thresholds (τ) for ARIMA/ML trade signals.

Ablation tests removes RSI, moving average, and position features to confirm feature relevance.

1.7 Future Work

Future work includes extending the RL framework to Double DQN and multi-ticker PPO training, integrating macroeconomic factors and sector rotations, and exploring hybrid supervised–RL architectures for risk-aware trading decisions.

References

[1] George S Atsalakis and Kimon P Valavanis. Surveying stock market forecasting techniques–part ii: Soft computing methods. *Expert Systems with Applications*, 36(3):5932–5941, 2009.

[2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.

[4] Tin Kam Ho. The random subspace method for constructing decision forests. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(8):832–844, 1998.

[5] Ehsan Hoseinzade and Saeed Haratizadeh. Cnnpred: Cnn-based stock market prediction using several data sources. *Expert Systems with Applications*, 129:273–285, 2019.

[6] Xiaoyang Li, Jian Ni, and Victor Chang. Application of deep reinforcement learning in stock trading strategies and stock forecasting. *Computing*, 102:1305–1322, 2020.

[7] John Moody and Matthew Saffell. Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4):875–889, 2001.

[8] Iqra Mushtaq Parray, Rabia Khaliq, Zahid Ali, Muhammad Waseem Anwar, and Shahrukh Khan. Time series data analysis of stock price movement using machine learning techniques. *Soft Computing*, 24:13409–13421, 2020.

[9] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.